

```
` `` python

import os

import sys

import asyncio

import aiofiles

import secrets

import schedule

import time

import tempfile

import shutil

from datetime import datetime, timedelta

from quart import Quart, request, jsonify, send_file

from quart_wtf import QuartForm, CSRFProtect

from prometheus_flask_exporter import PrometheusMetrics

from sqlalchemy import create_engine, text

import logging

import redis.asyncio as redis

from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes

from cryptography.hazmat.primitives import hashes, hmac

import webbrowser

import subprocess

import numpy as np

from sklearn.linear_model import LogisticRegression

from concurrent.futures import ThreadPoolExecutor

import boto3

import platform

# Настройка логирования
```

```
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s -
%(message)s')
```

```
# Глобальные объекты
```

```
REDIS = redis.Redis(host='localhost', port=6379, decode_responses=True)
```

```
EXECUTOR = ThreadPoolExecutor(max_workers=4)
```

```
KMS_CLIENT = boto3.client('kms', region_name='us-east-1') # Укажите ваш регион
```

```
# ===== Установщик библиотек
=====
```

```
INSTALL_LIBS = [
    ('quart', 'quart'),
    ('quart_wtf', 'quart-wtf'),
    ('aiofiles', 'aiofiles'),
    ('sqlalchemy', 'SQLAlchemy'),
    ('prometheus_flask_exporter', 'prometheus-flask-exporter'),
    ('psycopg2', 'psycopg2-binary'),
    ('redis', 'redis'),
    ('cryptography', 'cryptography'),
    ('sklearn', 'scikit-learn'),
    ('boto3', 'boto3') # Для KMS
]
```

```
def prompt_install():
```

```
    logging.info("="*60)
```

```
    logging.info("HybridCache — Установщик и первый запуск")
```

```
    logging.info("="*60)
```

```
    logging.info("\nДля работы HybridCache требуются Python-библиотеки:")
```

```
    logging.info(" Quart, quart-wtf, aiofiles, SQLAlchemy, prometheus-flask-exporter,
psycopg2, redis, cryptography, scikit-learn, boto3\n")
```

```

time.sleep(1)

missing = []

for lib, pkg in INSTALL_LIBS:

    try:

        __import__(lib)

        logging.info(f"✔ {lib} — установлено")

    except ImportError:

        logging.warning(f"✗ {lib} — не найдено")

        missing.append(pkg)

if missing:

    logging.info("Можно установить недостающие библиотеки автоматически.")

    choice = input(f"Установить их через pip? ({', '.join(missing)}) [Y/n]: ")

    if choice.lower() in ('', 'y', 'yes', 'д', 'да'):

        for pkg in missing:

            logging.info(f"Устанавливаем {pkg}...")

            try:

                subprocess.run([sys.executable, '-m', 'pip', 'install', pkg], check=True)

            except subprocess.CalledProcessError as e:

                logging.error(f"Ошибка установки {pkg}: {e}")

                input("Нажмите Enter для продолжения или Ctrl+C для выхода...")

            logging.info("Установка завершена, продолжаем!\n")

        else:

            logging.info("Пропуск автоматической установки. Установите вручную:\n")

            logging.info("pip install quart quart-wtf aiofiles SQLAlchemy prometheus-flask-  
exporter pycogp2-binary redis cryptography scikit-learn boto3\n")

            input("Нажмите Enter для продолжения...\n")

        else:

            logging.info("Все необходимые библиотеки уже установлены.\n")

            time.sleep(1)

```

```
prompt_install()
```

```
# ===== Конфигурация и профили  
=====
```

```
def choose_profile():
```

```
    logging.info("Выберите сценарий использования HybridCache:")
```

```
    logging.info("1. Корпоративный (офисы, базы данных, документооборот)")
```

```
    logging.info("2. Домашний ПК (фото, видео, музыка, игры)")
```

```
    logging.info("3. ПК для ИИ/ML (модели, датасеты, эксперименты)")
```

```
    profile = input("Введите номер профиля (1-3) или нажмите Enter для стандартных  
настроек: ")
```

```
    sys_platform = platform.system()
```

```
    if profile == "1":
```

```
        if sys_platform == "Linux":
```

```
            ssd_dir = "/var/cache/ssd"
```

```
            cold_dir = "/var/cache/cold"
```

```
        elif sys_platform == "Darwin":
```

```
            ssd_dir = "/Users/Shared/HybridCache/ssd"
```

```
            cold_dir = "/Users/Shared/HybridCache/cold"
```

```
        else: # Windows
```

```
            ssd_dir = "C:\\HybridCache\\ssd"
```

```
            cold_dir = "C:\\HybridCache\\cold"
```

```
    return {
```

```
        "ssd_cache_dir": ssd_dir,
```

```
        "cold_storage_dir": cold_dir,
```

```
        "max_ram_symlinks": 1000,
```

```
        "profile_name": "corporate",
```

```
        "profile_code": 1
```

```
    }
```

```
elif profile == "2":

    if sys_platform == "Linux":

        ssd_dir = "/home/user/HybridCache/ssd"

        cold_dir = "/home/user/HybridCache/cold"

    elif sys_platform == "Darwin":

        ssd_dir = "/Users/Shared/HybridCache/ssd"

        cold_dir = "/Users/Shared/HybridCache/cold"

    else: # Windows

        ssd_dir = "C:\\HybridCache\\ssd"

        cold_dir = "C:\\HybridCache\\cold"

    return {

        "ssd_cache_dir": ssd_dir,

        "cold_storage_dir": cold_dir,

        "max_ram_symlinks": 500,

        "profile_name": "home",

        "profile_code": 2

    }

elif profile == "3":

    if sys_platform == "Linux":

        ssd_dir = "/mnt/ssd/ssd"

        cold_dir = "/mnt/hdd/cold"

    elif sys_platform == "Darwin":

        ssd_dir = "/Users/Shared/HybridCache/ssd"

        cold_dir = "/Users/Shared/HybridCache/cold"

    else: # Windows

        ssd_dir = "D:\\HybridCache\\ssd"

        cold_dir = "E:\\HybridCache\\cold"

    return {

        "ssd_cache_dir": ssd_dir,
```

```

        "cold_storage_dir": cold_dir,

        "max_ram_symlinks": 2000,

        "profile_name": "ml",

        "profile_code": 3

    }
else:

    if sys_platform == "Linux":

        ssd_dir = "/var/cache/ssd"

        cold_dir = "/var/cache/cold"

    elif sys_platform == "Darwin":

        ssd_dir = "/Users/Shared/HybridCache/ssd"

        cold_dir = "/Users/Shared/HybridCache/cold"

    else: # Windows

        ssd_dir = "C:\\HybridCache\\ssd"

        cold_dir = "C:\\HybridCache\\cold"

    return {

        "ssd_cache_dir": ssd_dir,

        "cold_storage_dir": cold_dir,

        "max_ram_symlinks": 1000,

        "profile_name": "default",

        "profile_code": 0

    }

```

```
base_config = choose_profile()
```

```

CONFIG = {

    **base_config,

    "db_url": "postgresql://user:pass@localhost/hybridcache",

    "max_file_size_mb": 100,

    "min_ssd_free_mb": 500,

```

```
"enable_encryption": True,  
"kms_key_id": "alias/hybridcache-key", # ID мастер-ключа KMS  
"aws_region": "us-east-1" # Регион AWS  
}
```

```
logging.info(f"Используется профиль: {CONFIG['profile_name']}")
```

```
logging.info(f"SSD-кеш: {CONFIG['ssd_cache_dir']} — холодное хранилище:  
{CONFIG['cold_storage_dir']}\n")
```

```
# Проверка директорий и прав
```

```
for dir_path in [CONFIG["ssd_cache_dir"], CONFIG["cold_storage_dir"]]:
```

```
    if not os.path.exists(dir_path):
```

```
        os.makedirs(dir_path, exist_ok=True)
```

```
    if not os.access(dir_path, os.W_OK):
```

```
        logging.error(f"Нет прав на запись в {dir_path}")
```

```
        sys.exit(1)
```

```
# Проверка свободного места
```

```
def check_disk_space(path, min_mb=100):
```

```
    try:
```

```
        usage = shutil.disk_usage(path)
```

```
        free_mb = usage.free / 1024 / 1024
```

```
        if free_mb < min_mb:
```

```
            logging.warning(f"Недостаточно места на диске {path}: {free_mb:.2f} MB  
свободно")
```

```
            return False
```

```
            return True
```

```
    except Exception as e:
```

```
        logging.error(f"Ошибка проверки диска {path}: {e}")
```

```
        return False
```

```
# ===== Шифрование с KMS =====
```

```
async def generate_data_key(key):
```

```
    response = KMS_CLIENT.generate_data_key(
```

```
        KeyId=CONFIG["kms_key_id"],
```

```
        KeySpec='AES_256',
```

```
        EncryptionContext={'key': key}
```

```
    )
```

```
    return response['Plaintext'], response['CiphertextBlob'], secrets.token_bytes(16)
```

```
def encrypt_data(data, file_key, nonce):
```

```
    cipher = Cipher(algorithms.AES(file_key), modes.CTR(nonce))
```

```
    encryptor = cipher.encryptor()
```

```
    encrypted_data = encryptor.update(data) + encryptor.finalize()
```

```
    h = hmac.HMAC(file_key, hashes.SHA256())
```

```
    h.update(encrypted_data)
```

```
    hmac_tag = h.finalize()
```

```
    return encrypted_data + hmac_tag
```

```
def decrypt_data(data, file_key, nonce):
```

```
    hmac_tag = data[-32:] # HMAC-SHA256 — 32 байта
```

```
    encrypted_data = data[:-32]
```

```
    h = hmac.HMAC(file_key, hashes.SHA256())
```

```
    h.update(encrypted_data)
```

```
    h.verify(hmac_tag)
```

```
    cipher = Cipher(algorithms.AES(file_key), modes.CTR(nonce))
```

```
    decryptor = cipher.decryptor()
```

```
    return decryptor.update(encrypted_data) + decryptor.finalize()
```



```

async def store_key(key, encrypted_key, nonce):

    with engine.connect() as conn:

        conn.execute(

            text("INSERT INTO file_keys (key, encrypted_key, nonce) VALUES (:key,
:encrypted_key, :nonce) ON CONFLICT (key) DO NOTHING"),

            {"key": key, "encrypted_key": encrypted_key.hex(), "nonce": nonce.hex()}

        )

        conn.commit()


async def get_key(key):

    with engine.connect() as conn:

        result = conn.execute(text("SELECT encrypted_key, nonce FROM file_keys WHERE key =
:key"), {"key": key}).fetchone()

        if result:

            encrypted_key = bytes.fromhex(result[0])

            nonce = bytes.fromhex(result[1])

            try:

                response = KMS_CLIENT.decrypt(

                    CiphertextBlob=encrypted_key,

                    EncryptionContext={'key': key}

                )

                return response['Plaintext'], nonce

            except Exception as e:

                logging.error(f"Ошибка дешифрования ключа для {key}: {e}")

                return None, None

        return None, None


async def rotate_keys():

    logging.info("Запуск ротации ключей")

    for key in await get_symlink_keys():

```

```

symlink = await get_symlink(key)

if not symlink:
    continue

path = symlink["path"]

old_key, old_nonce = await get_key(key)

if not old_key:
    continue

new_key, new_encrypted_key, new_nonce = await generate_data_key(key)

async with aiofiles.open(path, "rb") as f:
    data = await f.read()

    decrypted = decrypt_data(data, old_key, old_nonce)

    encrypted = encrypt_data(decrypted, new_key, new_nonce)

async with aiofiles.open(path, "wb") as f:
    await f.write(encrypted)

await store_key(key, new_encrypted_key, new_nonce)

logging.info(f"Ключ для {key} обновлён")

```

```

# ===== Горячий кэш — Redis
=====

```

```

async def add_symlink(key, path, size):
    await REDIS.hset(f"symlink:{key}", mapping={
        "path": path,
        "size": str(size),
        "last_access": datetime.now().isoformat(),
        "access_count": "1"
    })

    keys = await REDIS.keys("symlink:*")

    if len(keys) > CONFIG["max_ram_symlinks"]:
        scores = []

```

for k in keys:

data = await REDIS.hgetall(k)

count = int(data["access_count"])

last_access = datetime.fromisoformat(data["last_access"])

score = count / (1 + (datetime.now() - last_access).total_seconds())

scores.append((k, score))

coldest = min(scores, key=lambda x: x[1])[0]

await REDIS.delete(coldest)

async def access_symlink(key):

if await REDIS.exists(f"symlink:{key}"):

async with REDIS.pipeline() as pipe:

pipe.hincrby(f"symlink:{key}", "access_count", 1)

pipe.hset(f"symlink:{key}", "last_access", datetime.now().isoformat())

await pipe.execute()

async def get_symlink(key):

data = await REDIS.hgetall(f"symlink:{key}")

if data:

return {

"path": data["path"],

"size": int(data["size"]),

"last_access": datetime.fromisoformat(data["last_access"]),

"access_count": int(data["access_count"])

}

return None

async def remove_symlink(key):

await REDIS.delete(f"symlink:{key}")

```
with engine.connect() as conn:
```

```
    conn.execute(text("DELETE FROM file_keys WHERE key = :key"), {"key": key})
```

```
    conn.commit()
```

```
async def get_symlink_keys():
```

```
    return [k.split(":", 1)[1] for k in await REDIS.keys("symlink:*")]
```

```
# ===== Тёплый кэш — SSD =====
```

```
async def ssd_put(key, data):
```

```
    if not check_disk_space(CONFIG["ssd_cache_dir"], CONFIG["min_ssd_free_mb"]):
```

```
        raise OSError("Недостаточно места на SSD")
```

```
    path = os.path.join(CONFIG["ssd_cache_dir"], key)
```

```
    if os.path.exists(path):
```

```
        raise FileExistsError(f"Файл {key} уже существует в SSD")
```

```
    if CONFIG["enable_encryption"]:
```

```
        file_key, encrypted_key, nonce = await generate_data_key(key)
```

```
        encrypted_data = encrypt_data(data, file_key, nonce)
```

```
        await store_key(key, encrypted_key, nonce)
```

```
    else:
```

```
        encrypted_data = data
```

```
    async with aiofiles.open(path, "wb") as f:
```

```
        await f.write(encrypted_data)
```

```
    await add_symlink(key, path, len(data))
```

```
async def ssd_get(key):
```

```
    entry = await get_symlink(key)
```

```
    path = os.path.join(CONFIG["ssd_cache_dir"], key)
```

```
    if entry and os.path.exists(entry["path"]):
```

```
        await access_symlink(key)
```

```

async with aiofiles.open(entry["path"], "rb") as f:
    data = await f.read()

if CONFIG["enable_encryption"]:
    file_key, nonce = await get_key(key)

    if not file_key:
        raise ValueError(f"Ключ для {key} не найден")

    return decrypt_data(data, file_key, nonce)

return data

elif os.path.exists(path):
    size = os.path.getsize(path)

    async with aiofiles.open(path, "rb") as f:
        data = await f.read()

    if CONFIG["enable_encryption"]:
        file_key, nonce = await get_key(key)

        if not file_key:
            raise ValueError(f"Ключ для {key} не найден")

        data = decrypt_data(data, file_key, nonce)

    await add_symlink(key, path, size)

    return data

return None

```

```

async def ssd_delete(key):
    path = os.path.join(CONFIG["ssd_cache_dir"], key)

    if os.path.exists(path):
        os.remove(path)

    await remove_symlink(key)

```

```

# ===== Холодный кэш — HDD
=====

```

```

async def cold_put(key, data):

    if not check_disk_space(CONFIG["cold_storage_dir"]):

        raise OSError("Недостаточно места на HDD")

    path = os.path.join(CONFIG["cold_storage_dir"], key)

    if os.path.exists(path):

        raise FileExistsError(f"Файл {key} уже существует в HDD")

    if CONFIG["enable_encryption"]:

        file_key, encrypted_key, nonce = await generate_data_key(key)

        encrypted_data = encrypt_data(data, file_key, nonce)

        await store_key(key, encrypted_key, nonce)

    else:

        encrypted_data = data

    async with aiofiles.open(path, "wb") as f:

        await f.write(encrypted_data)

    await add_symlink(key, path, len(data))


async def cold_get(key):

    path = os.path.join(CONFIG["cold_storage_dir"], key)

    if os.path.exists(path):

        async with aiofiles.open(path, "rb") as f:

            data = await f.read()

        if CONFIG["enable_encryption"]:

            file_key, nonce = await get_key(key)

            if not file_key:

                raise ValueError(f"Ключ для {key} не найден")

            return decrypt_data(data, file_key, nonce)

        return data

    return None

```

```

async def cold_delete(key):

    path = os.path.join(CONFIG["cold_storage_dir"], key)

    if os.path.exists(path):

        os.remove(path)

    await remove_symlink(key)


# ===== Кэширование cold-файлов на SSD
=====

async def promote_to_hot(key):

    cold_path = os.path.join(CONFIG["cold_storage_dir"], key)

    ssd_path = os.path.join(CONFIG["ssd_cache_dir"], key)

    if os.path.exists(cold_path) and not os.path.exists(ssd_path):

        if check_disk_space(CONFIG["ssd_cache_dir"], CONFIG["min_ssd_free_mb"]):

            shutil.move(cold_path, ssd_path)

            size = os.path.getsize(ssd_path)

            await add_symlink(key, ssd_path, size)

            update_stats(key, "hot")

            logging.info(f"Файл {key} перемещён из HDD в SSD")


# ===== ML-кэширование =====

ML_MODEL = LogisticRegression()


async def train_ml_model():

    logging.info("Обучение ML-модели для предиктивного кэширования")

    with engine.connect() as conn:

        result = conn.execute(text("SELECT key, location, access_count, last_access FROM
file_stats")).fetchall()

    X, y = [], []

    for key, location, access_count, last_access in result:

        days_since_last = (datetime.now() - datetime.fromisoformat(last_access)).days

```

```
size = 0

symlink = await get_symlink(key)

if symlink:
    size = symlink["size"]

X.append([access_count, days_since_last, size, CONFIG["profile_code"]])
y.append(1 if location == "hot" else 0)

if len(X) > 10:
    ML_MODEL.fit(X, y)

    logging.info("ML-модель обучена")

    await REDIS.delete("ml:*")
else:
    logging.warning("Недостаточно данных для обучения ML-модели")
```

```
async def predict_and_promote():
```

```
    logging.info("Запуск ML-прогноза для промоушна файлов")
```

```
    with engine.connect() as conn:
```

```
        result = conn.execute(text("SELECT key, access_count, last_access FROM file_stats
WHERE location = 'cold'")).fetchall()
```

```
tasks = []
```

```
async def process_file(key, access_count, last_access):
```

```
    cache_key = f"ml:{key}"
```

```
    cached_prob = await REDIS.get(cache_key)
```

```
    if cached_prob:
```

```
        prob = float(cached_prob)
```

```
    else:
```

```
        days_since_last = (datetime.now() - datetime.fromisoformat(last_access)).days
```

```
        size = 0
```

```
        symlink = await get_symlink(key)
```



```

if symlink:

    size = symlink["size"]

    features = [[access_count, days_since_last, size, CONFIG["profile_code"]]]

    prob = ML_MODEL.predict_proba(features)[0][1] if hasattr(ML_MODEL,
"predict_proba") else 0

    await REDIS.setex(cache_key, 3600, prob)

    load = len(await REDIS.keys("symlink:*")) / CONFIG["max_ram_symlinks"]

    threshold = 0.7 if load < 0.8 else 0.9

    if prob > threshold:

        tasks.append(asyncio.create_task(promote_to_hot(key)))

for key, access_count, last_access in result:

    await process_file(key, access_count, last_access)

if tasks:

    await asyncio.gather(*tasks)

logging.info(f"Завершён ML-прогноз, промоушено {len(tasks)} файлов")

# ===== Миграция файлов =====

def move_old_files():

    logging.info("Запуск миграции файлов из SSD в HDD")

    def move_file(key, entry):

        ssd_path = os.path.join(CONFIG["ssd_cache_dir"], key)

        cold_path = os.path.join(CONFIG["cold_storage_dir"], key)

        if os.path.exists(ssd_path) and not os.path.exists(cold_path):

            shutil.move(ssd_path, cold_path)

            asyncio.run(remove_symlink(key))

            update_stats(key, "cold")

            logging.info(f"Файл {key} перемещён из SSD в HDD")

```

```

keys = asyncio.run(get_symlink_keys())

futures = []

for key in keys:

    entry = asyncio.run(get_symlink(key))

    if entry and (datetime.now() - entry["last_access"]).days > 7:

        futures.append(EXECUTOR.submit(move_file, key, entry))

```

```

for future in futures:

    future.result()

```

```

# ===== PostgreSQL: статистика и ключи
=====

```

```

engine = create_engine(CONFIG["db_url"], pool_size=20, max_overflow=10)

```

```

def init_db():

    with engine.connect() as conn:

        conn.execute(text(

            """CREATE TABLE IF NOT EXISTS file_stats (

                key TEXT PRIMARY KEY,

                location TEXT,

                access_count INTEGER,

                last_access TIMESTAMP

            )"""

        ))

        conn.execute(text(

            """CREATE TABLE IF NOT EXISTS file_keys (

                key TEXT PRIMARY KEY,

                encrypted_key TEXT,

```

```

        nonce TEXT
    )"""

))

conn.execute(text("CREATE INDEX IF NOT EXISTS idx_location ON file_stats(location)"))

conn.execute(text("CREATE INDEX IF NOT EXISTS idx_access_count ON
file_stats(access_count)"))

conn.commit()

```

```

def clean_old_stats():

    threshold = (datetime.now() - timedelta(days=30)).isoformat()

    with engine.connect() as conn:

        conn.execute(text("DELETE FROM file_stats WHERE last_access < :threshold"),
{"threshold": threshold})

        conn.execute(text("DELETE FROM file_keys WHERE key NOT IN (SELECT key FROM
file_stats)"))

        conn.commit()

    logging.info("Очистка старых записей статистики и ключей завершена")

```

```

init_db()

schedule.every().week.at("03:00").do(clean_old_stats)

schedule.every().day.at("01:00").do(lambda: asyncio.run(train_ml_model()))

schedule.every().day.at("01:30").do(lambda: asyncio.run(predict_and_promote()))

schedule.every().month.do(lambda: asyncio.run(rotate_keys()))

```

```

def update_stats(key, location):

    with engine.connect() as conn:

        result = conn.execute(text("SELECT access_count FROM file_stats WHERE key = :key"),
{"key": key}).fetchone()

        acc = result[0] + 1 if result else 1

        conn.execute(

```

```

        text("INSERT INTO file_stats (key, location, access_count, last_access) "

        "VALUES (:key, :location, :access_count, :last_access) "

        "ON CONFLICT (key) UPDATE SET "

        "location = EXCLUDED.location, access_count = EXCLUDED.access_count,
last_access = EXCLUDED.last_access"),

        {"key": key, "location": location, "access_count": acc, "last_access":
datetime.now().isoformat()}

    )

    conn.commit()

def get_stats():

    with engine.connect() as conn:

        result = conn.execute(text("SELECT location, COUNT(*) FROM file_stats GROUP BY
location")).fetchall()

        return {loc: cnt for loc, cnt in result}

# ===== Quart, Limiter, Auth, Metrics
=====

app = Quart(__name__)

app.config['SECRET_KEY'] = secrets.token_hex(16)

csrf = CSRFProtect(app)

limiter = Limiter(app, key_func=lambda: request.remote_addr, default_limits=["100 per
minute"])

metrics = PrometheusMetrics(app)

AUTHORIZED_KEYS = {secrets.token_hex(16)}

FAILED_AUTH = {}

@app.before_request

async def check_auth():

    if request.endpoint in ("index", "api_stats", None):

        return

```

```

ip = request.remote_addr

key = request.headers.get("X-API-Key")

if key not in AUTHORIZED_KEYS:

    FAILED_AUTH[ip] = FAILED_AUTH.get(ip, 0) + 1

    if FAILED_AUTH[ip] > 10:

        return jsonify({"error": "IP blocked due to repeated unauthorized access"}), 403

    return jsonify({"error": "Unauthorized"}), 401

```

===== Web-интерфейс =====

```
@app.route("/")
```

```
async def index():
```

```
    hot_files = os.listdir(CONFIG["ssd_cache_dir"])
```

```
    cold_files = os.listdir(CONFIG["cold_storage_dir"])
```

```
    hot_list = "<ul>" + "".join(f"<li>{f}</li>" for f in hot_files if f != "hybridcache.db") + "</ul>"
```

```
    cold_list = "<ul>" + "".join(f"<li>{f}</li>" for f in cold_files) + "</ul>"
```

```
    form = QuartForm()
```

```
    csrf_token = form.csrf_token
```

```
    return f"""
```

```
<html>
```

```
<head><title>HybridCache (RAM-SSD-HDD)</title></head>
```

```
<body>
```

```
<h1>HybridCache ({CONFIG['profile_name']})</h1>
```

```
<h2>Горячие файлы (SSD):</h2>{hot_list}
```

```
<h2>Холодные файлы (HDD):</h2>{cold_list}
```

```
<form action='/api/put' method='post' enctype='multipart/form-data'>
```

```
    <input type='hidden' name='csrf_token' value='{csrf_token}'>
```

```
    <input type='text' name='key' placeholder='Имя файла' required>
```

```
    <input type='file' name='file' required>
```

```
    <select name='location'>
```

```

        <option value='hot'>Горячий (SSD + RAM-ярлык)</option>

        <option value='cold'>Холодный (HDD)</option>

    </select>

    <input type='submit' value='Добавить'>

</form>

<hr>

<form action='/api/get' method='get'>

    <input type='hidden' name='csrf_token' value='{csrf_token}'>

    <input type='text' name='key' placeholder='Имя для скачивания'>

    <input type='submit' value='Получить'>

</form>

<hr>

<form action='/api/delete' method='post'>

    <input type='hidden' name='csrf_token' value='{csrf_token}'>

    <input type='text' name='key' placeholder='Имя для удаления'>

    <select name='location'>

        <option value='hot'>Горячий</option>

        <option value='cold'>Холодный</option>

    </select>

    <input type='submit' value='Удалить'>

</form>

<hr>

<a href='/api/stats'>Статистика</a>

<div><b>API ключ:</b> <code>X-API-Key: {list(AUTHORIZED_KEYS)[0]}</code></div>

</body>

</html>

"""

```

```

@app.route("/api/put", methods=["POST"])

```

```

@limiter.limit("10/minute")

async def api_put():
    try:
        form = await request.form

        key = os.path.basename(form.get("key"))

        location = form.get("location", "hot")

        path = os.path.join(CONFIG["ssd_cache_dir" if location == "hot" else
"cold_storage_dir"], key)

        if os.path.exists(path):
            return jsonify({"status": "error", "message": f"Файл {key} уже существует."}), 409

        data = (await request.files)["file"].read()

        if len(data) > CONFIG["max_file_size_mb"] * 1024 * 1024:
            return jsonify({"status": "error", "message": "Слишком большой файл!"}), 413

        if location == "hot":
            await ssd_put(key, data)
            update_stats(key, "hot")
        else:
            await cold_put(key, data)
            update_stats(key, "cold")

        return jsonify({"status": "ok", "message": "Файл добавлен."})
    except FileExistsError as e:
        logging.error(f"Error in put: {e}")
        return jsonify({"status": "error", "message": str(e)}), 409
    except OSError as e:
        logging.error(f"Error in put: {e}")
        return jsonify({"status": "error", "message": str(e)}), 507
    except Exception as e:
        logging.error(f"Error in put: {e}")
        return jsonify({"status": "error", "message": str(e)}), 500

```

```

@app.route("/api/get", methods=["GET"])
@limiter.limit("30/minute")
async def api_get():
    try:
        key = os.path.basename(request.args.get("key"))
        data = await ssd_get(key)
        location = "hot"
        if data is None:
            data = await cold_get(key)
            location = "cold"
            if data is None:
                return jsonify({"status": "error", "message": "Файл не найден."}), 404
        with engine.connect() as conn:
            result = conn.execute(text("SELECT access_count FROM file_stats WHERE key = :key"), {"key": key}).fetchone()
            if result and result[0] > 10:
                await promote_to_hot(key)
        update_stats(key, location)
        with tempfile.NamedTemporaryFile(delete=False, suffix=f"_{key}") as tmp:
            tmp.write(data)
            tmp_path = tmp.name
        response = await send_file(tmp_path, as_attachment=True)
        os.remove(tmp_path)
        return response
    except Exception as e:
        logging.error(f"Error in get: {e}")
        return jsonify({"status": "error", "message": str(e)}), 500

```



```

@app.route("/api/delete", methods=["POST"])
@limiter.limit("10/minute")
async def api_delete():
    try:
        form = await request.form
        key = os.path.basename(form.get("key"))
        location = form.get("location", "hot")
        if location == "hot":
            await ssd_delete(key)
        else:
            await cold_delete(key)
        return jsonify({"status": "ok", "message": "Файл удалён."})
    except Exception as e:
        logging.error(f"Error in delete: {e}")
        return jsonify({"status": "error", "message": str(e)}), 500

```

```

@app.route("/api/stats", methods=["GET"])
async def api_stats():
    try:
        stats = get_stats()
        return jsonify(stats)
    except Exception as e:
        logging.error(f"Error in stats: {e}")
        return jsonify({"status": "error", "message": str(e)}), 500

```

===== Инструкции для продакшена
 =====

Для продакшена:

1. Установить Gunicorn: pip install gunicorn

2. Запустить: gunicorn -w 4 -b 127.0.0.1:8080
hybridcache_symlink_hotcache_project_Version10:app

3. Настроить Nginx (пример конфигурации):

```
# server {  
  
#     listen 443 ssl;  
  
#     server_name hybridcache.example.com;  
  
#     ssl_certificate /etc/ssl/cert.pem;  
  
#     ssl_certificate_key /etc/ssl/key.pem;  
  
#     location / {  
  
#         proxy_pass http://127.0.0.1:8080;  
  
#         proxy_set_header Host $host;  
  
#         proxy_set_header X-Real-IP $remote_addr;  
  
#     }  
  
# }
```

4. Unit-тесты с pytest:

```
# ```python  
  
# def test_ssd_put_get():  
  
#     asyncio.run(ssd_put("test.txt", b"data"))  
  
#     assert asyncio.run(ssd_get("test.txt")) == b"data"  
  
# ```
```

5. Dockerfile для контейнеризации:

```
# ```dockerfile  
  
# FROM python:3.9  
  
# WORKDIR /app  
  
# COPY . .  
  
# RUN pip install -r requirements.txt  
  
# CMD ["gunicorn", "-w", "4", "-b", "0.0.0.0:8080",  
"hybridcache_symlink_hotcache_project_Version10:app"]  
  
# ```
```

6. Настройка AWS KMS:

- Создать СМК в AWS KMS.

- Настроить IAM-политику для доступа к KMS.

- Указать KeyId в CONFIG["kms_key_id"].

===== Запуск сервера и планировщика
=====

if __name__ == "__main__":

import threading

threading.Thread(target=lambda: schedule.every().day.at("02:00").do(move_old_files),
daemon=True).start()

threading.Thread(target=lambda: app.run(host="127.0.0.1", port=8080),
daemon=True).start()

logging.info("HybridCache сервер запущен на http://127.0.0.1:8080")

webbrowser.open("http://127.0.0.1:8080")